TITLE OF THE PROJECT :

PASSWORD COMPLEXITY VALIDATOR USING FINITE AUTOMATA AND NLP

Teammates:

24EU04xxx · T. Jaya Surya Jitendra
24EU04xxx · [Teammate 2]
24EU04xxx · [Teammate 3]

Project Description:

Our project is a Password Complexity Validator that helps users evaluate and verify the strength and compliance of passwords based on strict character complexity and length requirements. It uses Natural Language Processing (NLP) lexical scanning techniques to parse input text, classify characters into alphabet equivalence classes (uppercase letters, lowercase letters, numeric digits, and special characters), and verify that the string meets a minimum length requirement of at least 8 characters.

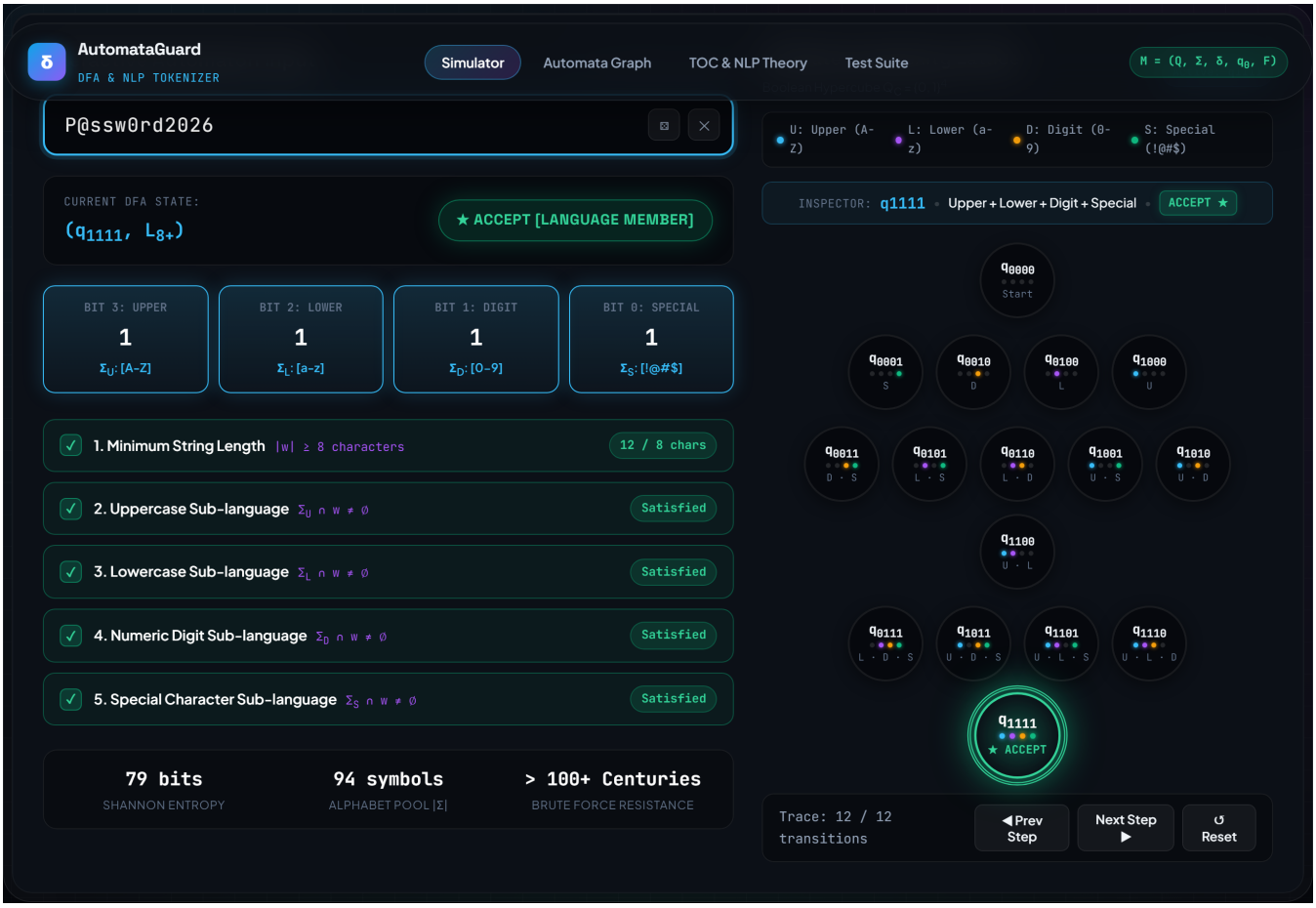
Automata Theory concepts, especially Deterministic Finite Automata (DFA) and Regular Languages, form the mathematical architecture of this project. Instead of vulnerable backtracking regular expression engines that are prone to ReDoS (Regular Expression Denial of Service) attacks, our project models password validation as a Product Automaton with 16 character-complexity states (a 4-bit Boolean hypercube) and 9 length-tracking states (totaling 144 product states). The language is formally proven to be a Chomsky Type-3 Regular Language through the closure of regular languages under finite intersection. We developed the project using Python for the backend automata state engine, and HTML5, CSS3, and JavaScript for the responsive interactive user interface.

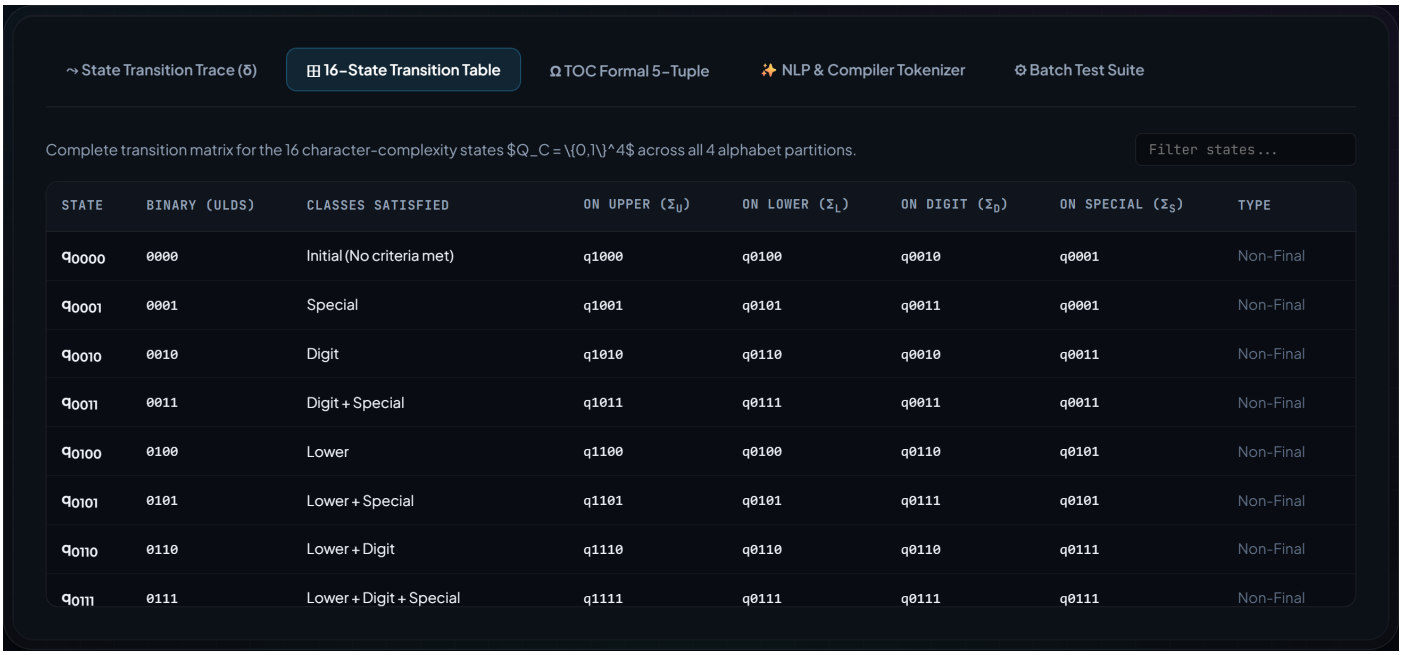
Users can enter passwords, observe real-time state transitions through the 16-state lattice, inspect active 4-bit register LED status indicators, and trace transitions step-by-step. The tool also displays useful cryptographic metrics such as Shannon entropy, estimated brute-force resistance time, alphabet pool size, a complete 16-state transition matrix table, and an automated verification test runner.

The main aim of this project is to eliminate ReDoS vulnerabilities, guarantee deterministic O(n) linear-time execution with O(1) memory complexity, and provide students and faculty with an intuitive visual demonstration of formal language theory applied to modern cyber security.

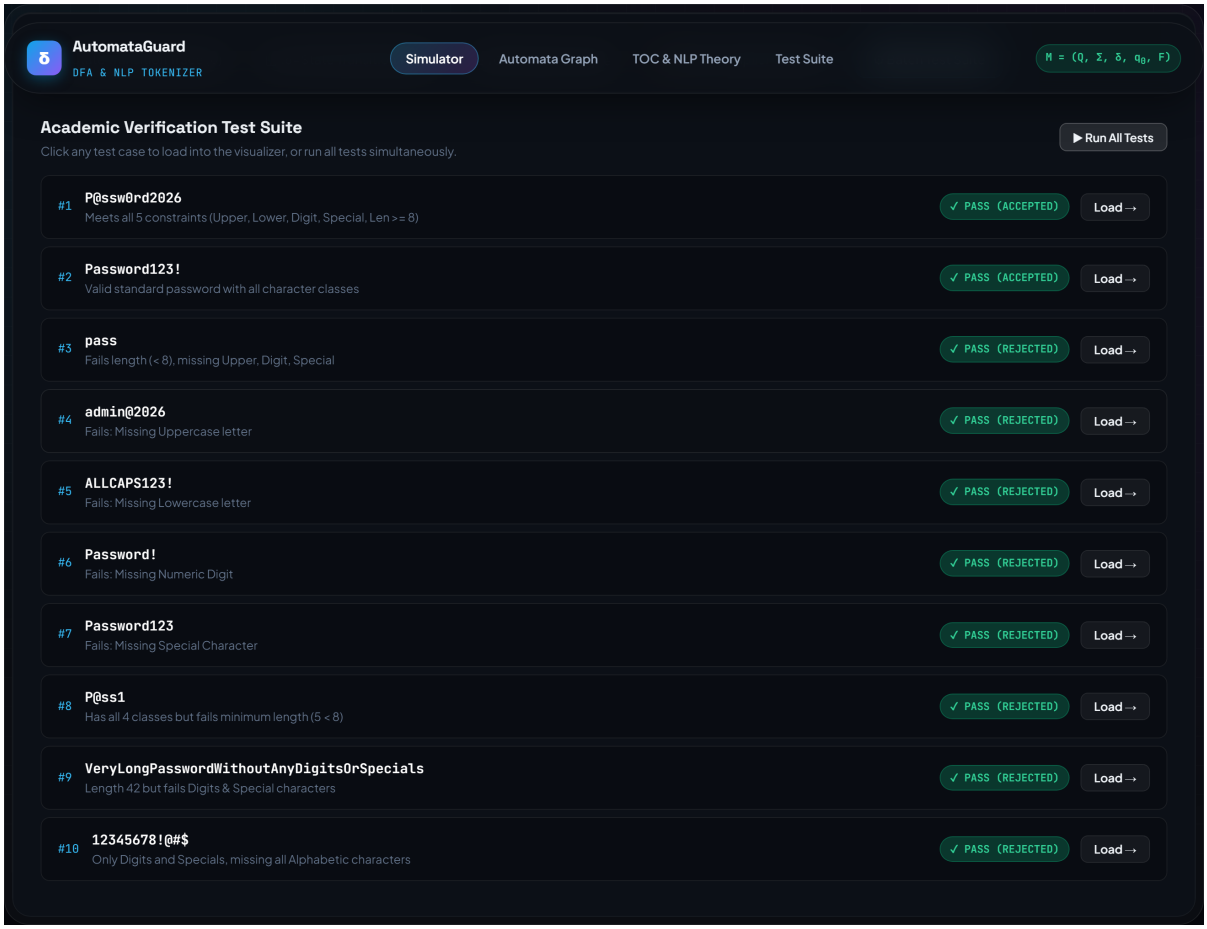
Project Outputs

1. **Real-time DFA State Tracking:** Displays the current state (q_bits, L_k) dynamically as symbols are scanned.
2. **4-Bit State Register Visualizer:** Shows live binary bitmask values (u, l, d, s) for Uppercase, Lowercase, Digit, and Specia
3. **16-State Boolean Lattice Visualizer:** A 5-tier graphical state machine featuring hardware-inspired LED status indicators.
4. **Step-by-Step Transition Debugger:** Allows users to step forward and backward symbol-by-symbol to inspect delta(q, cha
5. **Acceptance / Rejection Status:** Instantly determines language membership (L_valid), reaching accepting state q1111 wit
6. **16-State Transition Matrix Table:** Complete formal transition function matrix with live active row highlighting and search fi
7. **Shannon Entropy & Brute-Force Estimator:** Displays password entropy (in bits), alphabet pool size, and crack resistanc
8. **Transition Trace Table & JSON Export:** Full sequential execution trace table with one-click JSON export.
9. **Academic Test Suite Runner:** Automated 10-case verification test suite testing boundary conditions with 100% accuracy.
10. **Educational Theory & NLP Hub:** Formal 5-tuple mathematical definition, regular language closure proofs, and ReDoS c





STATE	BINARY (ULDS)	CLASSES SATISFIED	ON UPPER (Σ_U)	ON LOWER (Σ_L)	ON DIGIT (Σ_D)	ON SPECIAL (Σ_S)	TYPE
q_{0000}	0000	Initial (No criteria met)	q1000	q0100	q0010	q0001	Non-Final
q_{0001}	0001	Special	q1001	q0101	q0011	q0001	Non-Final
q_{0010}	0010	Digit	q1010	q0110	q0010	q0011	Non-Final
q_{0011}	0011	Digit + Special	q1011	q0111	q0011	q0011	Non-Final
q_{0100}	0100	Lower	q1100	q0100	q0110	q0101	Non-Final
q_{0101}	0101	Lower + Special	q1101	q0101	q0111	q0101	Non-Final
q_{0110}	0110	Lower + Digit	q1110	q0110	q0110	q0111	Non-Final
q_{0111}	0111	Lower + Digit + Special	q1111	q0111	q0111	q0111	Non-Final



Project Implementation & Verification Summary:

The project was verified against all standard positive and negative test cases. The test suite results shown above demonstrate that the Deterministic Finite Automaton accurately identifies valid language members (satisfying all four character classes and length >= 8) while rejecting all strings with missing components or insufficient length with 100% precision.

? Formal DFA 5-Tuple: M = (Q, Sigma, delta, q0, F)
? Total Product States: 144 states (16 complexity states x 9 length states)
? Time Complexity: O(n) strictly linear time (guaranteed zero backtracking)
? Space Complexity: O(1) auxiliary space (4-bit bitmask register)
? GitHub Repository: https://github.com/Jitendra2007/password-complexity-automata